

Assignment Number:12.3		
Q.No.	Question	Expected Time to complete
1	Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools Lab Objectives The objectives of this laboratory exercise are to: • Apply AI-assisted programming techniques to implement sorting and searching algorithms. • Analyze and compare algorithm efficiency using time and space complexity. • Understand how AI tools can suggest optimizations and alternative algorithmic approaches. • Strengthen problem-solving skills through real-world, data-driven scenarios. Learning Outcomes After completing this lab, students will be able to: • Implement and optimize classic algorithms using AI-assisted coding tools. • Compare multiple algorithms for the same problem and justify their selection. • Measure and analyze runtime performance using experimental data. • Critically review and refine AI-generated algorithmic solutions. Task 1: Sorting Student Records for Placement Drive Scenario SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order. Tasks 1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA). 2. Implement the following sorting algorithms using AI assistance: ○ Quick Sort ○ Merge Sort 3. Measure and compare runtime performance for large datasets. 4. Write a function to display the top 10 students based on CGPA. Expected Outcome • Correctly sorted student records. • Performance comparison between Quick Sort and Merge Sort. • Clear output of top-performing students.	Week3 - Wednesday

Prompt :

Create a Python program for SR University's Training and Placement Cell to store student records (Name, Roll Number, CGPA) and sort them by CGPA in descending order. Implement both Quick Sort and Merge Sort using AI assistance, then measure and compare their runtime performance on large datasets. Finally, write a function to display the top 10 students based on CGPA to highlight the best performers.

Code :

```
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __str__(self):
        return f"Name: {self.name}, Roll No: {self.roll_number}, CGPA: {self.cgpa}"

    def __lt__(self, other):
        if isinstance(other, Student):
            return self.cgpa < other.cgpa
        return NotImplemented

print("Student class defined successfully.")

... Student class defined successfully.

def quick_sort(students):
    if len(students) <= 1:
        return students

    pivot = students[len(students) // 2]
    less = [s for s in students if s.cgpa > pivot.cgpa] # Descending order
    equal = [s for s in students if s.cgpa == pivot.cgpa]
    greater = [s for s in students if s.cgpa < pivot.cgpa] # Descending order

    return quick_sort(less) + equal + quick_sort(greater)

print("quick_sort function defined successfully.")

... quick_sort function defined successfully.
```

```

▶ print("\n--- Testing Quick Sort ---")

# Create a list of Student objects
students = [
    Student("Alice", "A001", 3.8),
    Student("Bob", "B002", 3.5),
    Student("Charlie", "C003", 4.0),
    Student("David", "D004", 3.9),
    Student("Eve", "E005", 3.7)
]

print("Original list of students:")
for student in students:
    print(student)

# Sort the students using quick_sort
sorted_students = quick_sort(students)

print("\nSorted list of students (descending CGPA):")
for student in sorted_students:
    print(student)

print("--- Quick Sort testing complete ---")

```

```

...
--- Testing Quick Sort ---
Original list of students:
Name: Alice, Roll No: A001, CGPA: 3.8

```

```

--- Testing Quick Sort ---
Original list of students:
Name: Alice, Roll No: A001, CGPA: 3.8
Name: Bob, Roll No: B002, CGPA: 3.5
Name: Charlie, Roll No: C003, CGPA: 4.0
Name: David, Roll No: D004, CGPA: 3.9
Name: Eve, Roll No: E005, CGPA: 3.7

Sorted list of students (descending CGPA):
Name: Charlie, Roll No: C003, CGPA: 4.0
Name: David, Roll No: D004, CGPA: 3.9
Name: Alice, Roll No: A001, CGPA: 3.8
Name: Eve, Roll No: E005, CGPA: 3.7
Name: Bob, Roll No: B002, CGPA: 3.5
--- Quick Sort testing complete ---

```

```

def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left_half = students[:mid]
    right_half = students[mid:]

    left_half = merge_sort(left_half)
    right_half = merge_sort(right_half)

    return merge(left_half, right_half)

def merge(left, right):
    merged_list = []
    left_index = 0
    right_index = 0

    while left_index < len(left) and right_index < len(right):
        # Sort in descending order based on cgpa
        if left[left_index].cgpa >= right[right_index].cgpa:
            merged_list.append(left[left_index])
            left_index += 1
        else:
            merged_list.append(right[right_index])
            right_index += 1

    # Append any remaining elements from left_half
    merged_list.extend(left[left_index:])

    print("Original list of students (for Merge Sort):")
    for student in students_merge:
        print(student)

    # Sort the students using merge_sort
    sorted_students_merge = merge_sort(students_merge)

    print("\nSorted list of students (descending CGPA via Merge Sort):")
    for student in sorted_students_merge:
        print(student)

    print("--- Merge Sort testing complete ---")

...

--- Testing Merge Sort ---
Original list of students (for Merge Sort):
Name: Alice, Roll No: A001, CGPA: 3.8
Name: Bob, Roll No: B002, CGPA: 3.5
Name: Charlie, Roll No: C003, CGPA: 4.0
Name: David, Roll No: D004, CGPA: 3.9
Name: Eve, Roll No: E005, CGPA: 3.7

Sorted list of students (descending CGPA via Merge Sort):
Name: Charlie, Roll No: C003, CGPA: 4.0
Name: David, Roll No: D004, CGPA: 3.9
Name: Alice, Roll No: A001, CGPA: 3.8
Name: Eve, Roll No: E005, CGPA: 3.7
Name: Bob, Roll No: B002, CGPA: 3.5
--- Merge Sort testing complete ---

```

```

import random
import string

def generate_students(num_students):
    students_data = []
    for i in range(num_students):
        # Generate random name
        name_length = random.randint(5, 10)
        name = ''.join(random.choice(string.ascii_letters) for _ in range(name_length)).capitalize()

        # Generate unique roll number
        roll_number = f"R{i+1:05d}" # e.g., R00001, R00002

        # Generate random CGPA between 2.0 and 4.0
        cgpa = round(random.uniform(2.0, 4.0), 2)

        students_data.append(Student(name, roll_number, cgpa))
    return students_data

# Generate a large dataset
large_student_dataset = generate_students(50000)

print(f"Generated {len(large_student_dataset)} student records.")

... Generated 50000 student records.

import time
import copy

def measure_sorting_performance(sort_function, dataset, repetitions=5):
    runtimes = []
    print(f"\nMeasuring performance for {sort_function.__name__}...")
    for i in range(repetitions):
        # Create a fresh copy of the original dataset for each run
        # to ensure both algorithms sort the same initial data.
        data_copy = copy.deepcopy(dataset)

        start_time = time.time()
        _ = sort_function(data_copy)
        end_time = time.time()
        duration = end_time - start_time
        runtimes.append(duration)
        print(f" Repetition {i+1}: {duration:.4f} seconds")

    average_runtime = sum(runtimes) / repetitions
    print(f"Average runtime for {sort_function.__name__} over {repetitions} repetitions: {average_runtime:.4f} seconds")
    return average_runtime

# Measure Quick Sort performance
quick_sort_avg_time = measure_sorting_performance(quick_sort, large_student_dataset, repetitions=5)

# Measure Merge Sort performance
merge_sort_avg_time = measure_sorting_performance(merge_sort, large_student_dataset, repetitions=5)

print("\n--- Runtime performance comparison complete ---")

Measuring performance for quick_sort...
Repetition 1: 0.3259 seconds
Repetition 2: 0.2099 seconds
Repetition 3: 0.2130 seconds
Repetition 4: 0.2169 seconds
Repetition 5: 0.2463 seconds
Average runtime for quick_sort over 5 repetitions: 0.2424 seconds

Measuring performance for merge_sort...
Repetition 1: 0.2586 seconds
Repetition 2: 0.2754 seconds
Repetition 3: 0.2858 seconds
Repetition 4: 0.2978 seconds
Repetition 5: 0.2883 seconds
Average runtime for merge_sort over 5 repetitions: 0.2812 seconds

--- Runtime performance comparison complete ---

```

- Explanation : A Student class was successfully defined with name, roll_number, and cgpa attributes, including a __str__ method for display and a __lt__ method for sorting based on cgpa in ascending order (though the sorting algorithms implemented custom logic for descending order). Both Quick Sort and Merge Sort algorithms were successfully implemented to sort a list of Student objects in descending order by cgpa.

Task 2: Implementing Bubble Sort with AI Comments

- **Task:** Write a Python implementation of **Bubble Sort**.
- **Instructions:**
 - Students implement Bubble Sort normally.
 - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
 - Request AI to provide **time complexity analysis**.
- **Expected Output:**
 - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt :

Create a Python program for SR University's Training and Placement Cell to store student records (Name, Roll Number, CGPA) and sort them by CGPA in descending order. Implement both Quick Sort and Merge Sort using AI assistance, then measure and compare their runtime performance on large datasets. Finally, write a function to display the top 10 students based on CGPA to highlight the best performers.

Code :

```
def bubble_sort(arr):  
    n = len(arr)  
    # Flag to optimize: if no two elements were swapped by inner loop, then break  
    # This means the array is already sorted  
    for i in range(n - 1): # Outer loop for passes  
        swapped = False # Initialize swapped flag for each pass  
        # Each pass moves the largest unsorted element to its correct final position  
        for j in range(0, n - i - 1): # Inner loop for comparisons and swaps  
            # Compare adjacent elements  
            if arr[j] > arr[j + 1]:  
                # If the current element is greater than the next, swap them  
                # This ensures that larger elements "bubble up" to the end of the array  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
                swapped = True # Mark that a swap occurred in this pass  
        # If no elements were swapped in the inner loop, the array is sorted  
        # This is the termination condition for the optimized version  
        if not swapped:  
            break  
    return arr  
  
print("Bubble Sort function defined successfully.")  
  
# Test the implementation  
example_list = [64, 34, 25, 12, 22, 11, 90]  
print(f"Original list: {example_list}")  
sorted_list = bubble_sort(example_list)  
print(f"Sorted list: {sorted_list}")
```

```

example_list_2 = [5, 1, 4, 2, 8]
print(f"Original list 2: {example_list_2}")
sorted_list_2 = bubble_sort(example_list_2)
print(f"Sorted list 2: {sorted_list_2}")

example_list_3 = [1, 2, 3, 4, 5] # Already sorted
print(f"Original list 3: {example_list_3}")
sorted_list_3 = bubble_sort(example_list_3)
print(f"Sorted list 3: {sorted_list_3}")

```

```

Bubble Sort function defined successfully.
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Original list 2: [5, 1, 4, 2, 8]
Sorted list 2: [1, 2, 4, 5, 8]
Original list 3: [1, 2, 3, 4, 5]
Sorted list 3: [1, 2, 3, 4, 5]

```

Explanation : Occurs when the array is already sorted. The outer loop runs once, and the inner loop makes $n-1$ comparisons. The swapped flag remains False, causing the algorithm to terminate after the first pass.

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion.
- **Instructions:**
 - Provide AI with partially completed functions for recursion.
 - Ask AI to complete the missing logic and add docstrings.
 - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
 - Working Quick Sort and Merge Sort implementations.
 - AI-generated explanation of average, best, and worst-case complexities.

Prompt :

Implement recursive Quick Sort and Merge Sort in Python, ask AI to complete missing logic with docstrings, then compare both on random, sorted, and reverse-sorted lists, and explain their average, best, and worst-case complexities.

Code :


```

def _partition(arr, low, high):
    """
    Helper function to partition the array for Quick Sort.

    This function takes the last element as a pivot, places the pivot element
    at its correct position in sorted array, and places all smaller (or equal)
    to left of pivot and all greater elements to right of pivot.

    Args:
        arr (list): The list of elements (Student objects) to be partitioned.
        low (int): The starting index of the sub-array to be partitioned.
        high (int): The ending index of the sub-array to be partitioned (inclusive).

    Returns:
        int: The final index of the pivot element after partitioning.
    """
    pivot = arr[high].cgpa
    i = low - 1 # Index of smaller element

    for j in range(low, high):
        # If current element is greater than or equal to pivot (for descending sort)
        if arr[j].cgpa >= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # Swap elements

    arr[i + 1], arr[high] = arr[high], arr[i + 1] # Swap pivot to its correct position
    return i + 1

```

```

def quick_sort(arr, low=0, high=None):
    """
    Implements the Quick Sort algorithm recursively to sort a list of Student objects
    in descending order based on their CGPA.

    Args:
        arr (list): The list of Student objects to be sorted.
        low (int, optional): The starting index of the sub-array to be sorted. Defaults to 0.
        high (int, optional): The ending index of the sub-array to be sorted (inclusive).
        Defaults to the last element's index.
    """
    if high is None:
        high = len(arr) - 1

    if low < high:
        # pi is partitioning index, arr[pi] is now at right place
        pi = _partition(arr, low, high)

        # Recursively sort elements before partition and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

    print("quick_sort function defined successfully with _partition helper.")

```

```

... quick_sort function defined successfully with _partition helper.

```



```

import random

def generate_random_list(size):
    """
    Generates a list of random integers.

    Args:
        size (int): The number of integers to generate.

    Returns:
        list: A list containing `size` random integers.
    """
    return [random.randint(0, 1000000) for _ in range(size)]

def generate_sorted_list(size):
    """
    Generates a list of integers sorted in ascending order.

    Args:
        size (int): The number of integers to generate.

    Returns:
        list: A list containing `size` integers in ascending order.
    """
    return list(range(size))

def generate_reverse_sorted_list(size):
    """
    Generates a list of integers sorted in descending order.

    Args:
        size (int): The number of integers to generate.

    Returns:
        list: A list containing `size` integers in descending order.
    """
    return list(range(size - 1, -1, -1))

# Generate large datasets
dataset_size = 50000

random_data = generate_random_list(dataset_size)
sorted_data = generate_sorted_list(dataset_size)
reverse_sorted_data = generate_reverse_sorted_list(dataset_size)

print(f"Generated random list with {len(random_data)} elements.")
print(f"Generated sorted list with {len(sorted_data)} elements.")
print(f"Generated reverse-sorted list with {len(reverse_sorted_data)} elements.")

... Generated random list with 50000 elements.
Generated sorted list with 50000 elements.
Generated reverse-sorted list with 50000 elements.

```

```

def merge_sort(arr):
    """
    Implements the Merge Sort algorithm recursively to sort a list of elements
    in descending order based on a comparison criterion.

    Args:
        arr (list): The list of elements to be sorted.

    Returns:
        list: The sorted list of elements in descending order.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    left_half = merge_sort(left_half)
    right_half = merge_sort(right_half)

    return merge(left_half, right_half)

def merge(left, right):
    """
    Helper function to merge two sorted lists into a single sorted list
    in descending order.

    Args:
        left (list): The first sorted list (left half).
        right (list): The second sorted list (right half).

    Returns:
        list: A new list containing all elements from 'left' and 'right'
        merged and sorted in descending order.
    """
    merged_list = []
    left_index = 0
    right_index = 0

    while left_index < len(left) and right_index < len(right):
        # For descending order, append the larger element
        # Assuming elements are comparable, e.g., have a .cgpa attribute for Student objects.
        # This assumes the comparison is direct for now, adjust if complex objects are used.
        if left[left_index] >= right[right_index]: # Compare directly for generic elements
            merged_list.append(left[left_index])
            left_index += 1
        else:
            merged_list.append(right[right_index])
            right_index += 1

    # Append any remaining elements from left_half
    while left_index < len(left):
        merged_list.append(left[left_index])
        left_index += 1

    # Append any remaining elements from right_half
    while right_index < len(right):
        merged_list.append(right[right_index])
        right_index += 1

    return merged_list

print("merge_sort and merge functions defined successfully.")

merge_sort and merge functions defined successfully.

```

```

def _partition(arr, low, high):
    """
    Helper function to partition the array for Quick Sort.

    This function takes the last element as a pivot, places the pivot element
    at its correct position in sorted array, and places all smaller (or equal)
    to left of pivot and all greater elements to right of pivot.

    Args:
        arr (list): The list of elements (Student objects) to be partitioned.
        low (int): The starting index of the sub-array to be partitioned.
        high (int): The ending index of the sub-array to be partitioned (inclusive).

    Returns:
        int: The final index of the pivot element after partitioning.
    """
    pivot = arr[high].cgpa
    i = low - 1 # Index of smaller element

    for j in range(low, high):
        # If current element is greater than or equal to pivot (for descending sort)
        if arr[j].cgpa >= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # Swap elements

    arr[i + 1], arr[high] = arr[high], arr[i + 1] # Swap pivot to its correct position
    return i + 1

def quick_sort(arr, low=0, high=None):
    """
    Implements the Quick Sort algorithm recursively to sort a list of Student objects
    in descending order based on their CGPA.

    Args:
        arr (list): The list of Student objects to be sorted.
        low (int, optional): The starting index of the sub-array to be sorted. Defaults to 0.
        high (int, optional): The ending index of the sub-array to be sorted (inclusive).
        Defaults to the last element's index.
    """
    if high is None:
        high = len(arr) - 1

    if low < high:
        # pi is partitioning index, arr[pi] is now at right place
        pi = _partition(arr, low, high)

        # Recursively sort elements before partition and after partition
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

    print("quick_sort function defined successfully with _partition helper.")
... quick_sort function defined successfully with _partition helper.

```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.
- Prompt :

Design a real-time inventory management system for a retail store with thousands of products, each having attributes like product ID, name, price, and stock quantity. Staff must quickly search products by ID or name and sort them by price or quantity for stock analysis. Recommend the most efficient search and sort algorithms, implement them in Python, and justify choices based on dataset size, update frequency, and performance requirements. Output should include a table mapping operations to algorithms with justification, plus working Python functions for searching and sorting.

Code :

```
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def __str__(self):
        return f"Product ID: {self.product_id}, Name: {self.name}, Price: ${self.price:.2f}, Stock: {self.stock_quantity}"

    def __lt__(self, other):
        if isinstance(other, Product):
            return self.product_id < other.product_id
        return NotImplemented

    def __le__(self, other):
        if isinstance(other, Product):
            return self.product_id <= other.product_id
        return NotImplemented

    def __gt__(self, other):
        if isinstance(other, Product):
            return self.product_id > other.product_id
        return NotImplemented

    def __ge__(self, other):
        if isinstance(other, Product):
            return self.product_id >= other.product_id
        return NotImplemented

    def __eq__(self, other):
        if isinstance(other, Product):
            return self.product_id == other.product_id
        return NotImplemented

    def __ne__(self, other):
        if isinstance(other, Product):
            return self.product_id != other.product_id
        return NotImplemented

print("Product class defined successfully.")

Product class defined successfully.
```

```

class InventoryManager:
    def __init__(self):
        self.products = {}
        print("InventoryManager initialized.")

    def add_product(self, product):
        self.products[product.product_id] = product
        print(f"Added product: {product.name} (ID: {product.product_id})")

    def get_product_by_id(self, product_id):
        product = self.products.get(product_id)
        if product:
            print(f"Found product: {product.name} (ID: {product_id})")
            return product
        else:
            print(f"Product with ID {product_id} not found.")
            return None

print("InventoryManager class defined successfully.")
... InventoryManager class defined successfully.

```

```

print("\n--- Testing InventoryManager ---")
inventory_manager = InventoryManager()

# Create some Product objects
product1 = Product("P001", "Laptop", 1200.00, 50)
product2 = Product("P002", "Mouse", 25.00, 200)
product3 = Product("P003", "Keyboard", 75.00, 100)

# Add products to the inventory
inventory_manager.add_product(product1)
inventory_manager.add_product(product2)
inventory_manager.add_product(product3)

print("\n--- Retrieving products by ID ---")

# Retrieve existing products
found_product = inventory_manager.get_product_by_id("P001")
if found_product:
    print(f"Retrieved: {found_product}")

found_product = inventory_manager.get_product_by_id("P003")
if found_product:
    print(f"Retrieved: {found_product}")

# Attempt to retrieve a non-existent product
found_product = inventory_manager.get_product_by_id("P004")
if found_product:

```

```

if found_product:
    print(f"Retrieved: {found_product}")

print("--- InventoryManager testing complete ---")

...
--- Testing InventoryManager ---
InventoryManager initialized.
Added product: Laptop (ID: P001)
Added product: Mouse (ID: P002)
Added product: Keyboard (ID: P003)

--- Retrieving products by ID ---
Found product: Laptop (ID: P001)
Retrieved: Product ID: P001, Name: Laptop, Price: $1200.00, Stock: 50
Found product: Keyboard (ID: P003)
Retrieved: Product ID: P003, Name: Keyboard, Price: $75.00, Stock: 100
Product with ID P004 not found.
--- InventoryManager testing complete ---

```

```

def _partition_products(arr, low, high, attribute):
    """
    Helper function to partition the array for Quick Sort based on a product attribute.

    This function takes the last element as a pivot, places the pivot element
    at its correct position in sorted array, and places all products with
    attribute values greater than or equal to the pivot to the left of pivot,
    and all smaller values to the right of the pivot (for descending sort).

    Args:
        arr (list): The list of Product objects to be partitioned.
        low (int): The starting index of the sub-array to be partitioned.
        high (int): The ending index of the sub-array to be partitioned (inclusive).
        attribute (str): The name of the attribute to sort by (e.g., 'price', 'stock_quantity').

    Returns:
        int: The final index of the pivot element after partitioning.
    """
    pivot_value = getattr(arr[high], attribute)
    i = low - 1 # Index of smaller element

    for j in range(low, high):
        # If current element's attribute value is greater than or equal to pivot's (for descending sort)
        if getattr(arr[j], attribute) >= pivot_value:
            i += 1
            arr[i], arr[j] = arr[j], arr[i] # Swap elements

```

```

    return i + 1

def quick_sort_products(arr, attribute, low=0, high=None):
    """
    Implements the Quick Sort algorithm recursively to sort a list of Product objects
    in descending order based on a specified attribute.

    Args:
        arr (list): The list of Product objects to be sorted.
        attribute (str): The name of the attribute to sort by (e.g., 'price', 'stock_quantity').
        low (int, optional): The starting index of the sub-array to be sorted. Defaults to 0.
        high (int, optional): The ending index of the sub-array to be sorted (inclusive).
        Defaults to the last element's index.
    """
    if high is None:
        high = len(arr) - 1

    if low < high:
        # pi is partitioning index, arr[pi] is now at right place
        pi = _partition_products(arr, low, high, attribute)

        # Recursively sort elements before partition and after partition
        quick_sort_products(arr, attribute, low, pi - 1)
        quick_sort_products(arr, attribute, pi + 1, high)

    print("quick_sort_products function defined successfully with _partition_products helper.")

```

Explanation : Both **Quick Sort (quick_sort_products)** and **Merge Sort (merge_sort_products)** algorithms were implemented to sort lists of Product objects in descending order based on specified attributes (e.g., 'price', 'stock_quantity'). Both algorithms demonstrate an average time complexity of

$O(N \log N)$, making them efficient for sorting larger datasets. Quick Sort offers generally faster average performance due to lower constant factors, while Merge Sort guarantees $O(N \log N)$ performance in all cases and is a stable sort, though it requires $O(N)$ auxiliary space.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (`sorted()`, dict lookups) and analyze trade-offs.

Real-Time Stock Data Sorting & Searching Scenario: An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently. • Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price). • Implement sorting algorithms to rank stocks by percentage change. • Implement a search function that retrieves stock data instantly when a stock symbol is entered. • Optimize sorting with Heap Sort and searching with Hash Maps. • Compare performance with standard library functions (`sorted()`, dict lookups) and analyze trade-offs

Code :

```
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price

    def calculate_percentage_change(self):
        if self.opening_price == 0:
            return 0.0 # Avoid division by zero
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100

    def __str__(self):
        percentage_change = self.calculate_percentage_change()
        return f"Symbol: {self.symbol}, Open: ${self.opening_price:.2f}, Close: ${self.closing_price:.2f}, Change: {percentage_change:.2f}%"

print("Stock class defined successfully.")

... Stock class defined successfully.
```



```

▶ print("\n--- Testing Stock Class ---")

# Create a Stock object
stock1 = Stock("AAPL", 150.00, 155.50)

# Print the stock object to test the __str__ method and percentage change calculation
print(stock1)

# Test with a different stock and values
stock2 = Stock("GOOG", 2500.00, 2450.00)
print(stock2)

# Test edge case: opening price is zero
stock3 = Stock("ZERO", 0.00, 10.00)
print(stock3)

print("--- Stock Class testing complete ---")

```

```

...
--- Testing Stock Class ---
Symbol: AAPL, Open: $150.00, Close: $155.50, Change: 3.67%
Symbol: GOOG, Open: $2500.00, Close: $2450.00, Change: -2.00%
Symbol: ZERO, Open: $0.00, Close: $10.00, Change: 0.00%
--- Stock Class testing complete ---

```

```

▶ import random
import string

def generate_synthetic_stock_data(num_stocks):
    stock_data = []
    for i in range(num_stocks):
        # Generate random symbol (3-5 uppercase letters)
        symbol_length = random.randint(3, 5)
        symbol = ''.join(random.choice(string.ascii_uppercase) for _ in range(symbol_length))

        # Generate random opening price (between 50 and 2000)
        opening_price = round(random.uniform(50.00, 2000.00), 2)

        # Generate closing price (within +/- 10% of opening price)
        change_factor = random.uniform(0.9, 1.1) # -10% to +10%
        closing_price = round(opening_price * change_factor, 2)

        stock_data.append(Stock(symbol, opening_price, closing_price))
    return stock_data

# Generate a large dataset for testing (e.g., 10,000 stocks)
large_stock_dataset = generate_synthetic_stock_data(10000)

print(f"Generated {len(large_stock_dataset)} synthetic stock records.")

```

```

... Generated 10000 synthetic stock records.

```

```

def _heapify(arr, n, i):
    """
    Heapifies a subtree rooted at index `i` in an array `arr` of size `n`.
    Maintains the max-heap property for sorting in descending order based on `percentage_change`.

    Args:
        arr (list): The list of Stock objects representing the heap.
        n (int): The size of the heap (number of elements in the current array segment).
        i (int): The root index of the subtree to be heapified.
    """
    largest = i # Initialize largest as root
    l = 2 * i + 1 # left child
    r = 2 * i + 2 # right child

    # See if left child of root exists and is greater than root
    if l < n and arr[l].calculate_percentage_change() > arr[largest].calculate_percentage_change():
        largest = l

    # See if right child of root exists and is greater than current largest
    if r < n and arr[r].calculate_percentage_change() > arr[largest].calculate_percentage_change():
        largest = r

    # Change root, if needed
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i] # swap

        # Heapify the root.

```

```

def heap_sort(arr):
    """
    Implements the Heap Sort algorithm to sort a list of Stock objects
    in descending order based on their `percentage_change` attribute.

    Args:
        arr (list): The list of Stock objects to be sorted.
    """
    n = len(arr)

    # Build a maxheap. The loop runs from (n//2 - 1) down to 0.
    # This is the last non-leaf node, so we heapify from here upwards.
    for i in range(n // 2 - 1, -1, -1):
        _heapify(arr, n, i)

    # One by one extract elements
    for i in range(n - 1, 0, -1):
        # Move current root to end
        arr[i], arr[0] = arr[0], arr[i] # swap
        # call max heapify on the reduced heap
        _heapify(arr, i, 0)

    print("heap_sort function defined successfully.")

... heap_sort function defined successfully.

print(
    "\n--- Testing Heap Sort Algorithm ---"
)

# Create a sample list of Stock objects
sample_stocks = [
    Stock("MSFT", 300.00, 305.00), # +1.67%
    Stock("AMZN", 100.00, 95.00), # -5.00%
    Stock("NVDA", 200.00, 220.00), # +10.00%
    Stock("TSLA", 700.00, 680.00), # -2.86%
    Stock("GOOG", 150.00, 160.00) # +6.67%
]

print("Original list of stocks:")
for stock in sample_stocks:
    print(f" {stock}, Change: {stock.calculate_percentage_change():.2f}%")

# Sort the stocks using heap_sort
heap_sort(sample_stocks)

print("\nSorted list of stocks (descending by percentage change):")
for stock in sample_stocks:
    print(f" {stock}, Change: {stock.calculate_percentage_change():.2f}%")

print(
    "--- Heap Sort Algorithm testing complete ---"
)

```

```

--- Testing Heap Sort Algorithm ---
Original list of stocks:
Symbol: MSFT, Open: $300.00, Close: $305.00, Change: 1.67%, Change: 1.67%
Symbol: AMZN, Open: $100.00, Close: $95.00, Change: -5.00%, Change: -5.00%
Symbol: NVDA, Open: $200.00, Close: $220.00, Change: 10.00%, Change: 10.00%
Symbol: TSLA, Open: $700.00, Close: $680.00, Change: -2.86%, Change: -2.86%
Symbol: GOOG, Open: $150.00, Close: $160.00, Change: 6.67%, Change: 6.67%

Sorted list of stocks (descending by percentage change):
Symbol: AMZN, Open: $100.00, Close: $95.00, Change: -5.00%, Change: -5.00%
Symbol: TSLA, Open: $700.00, Close: $680.00, Change: -2.86%, Change: -2.86%
Symbol: MSFT, Open: $300.00, Close: $305.00, Change: 1.67%, Change: 1.67%
Symbol: GOOG, Open: $150.00, Close: $160.00, Change: 6.67%, Change: 6.67%
Symbol: NVDA, Open: $200.00, Close: $220.00, Change: 10.00%, Change: 10.00%
--- Heap Sort Algorithm testing complete ---

```

Explanation : A Student class was successfully defined with name, roll_number, and cgpa attributes, including a __str__ method for display and a __lt__ method for sorting based on cgpa in ascending order

	(though the sorting algorithms implemented custom logic for descending order). Both Quick Sort and Merge Sort algorithms were successfully implemented to sort a list of Student objects in descending order by cgpa.	
--	--	--